

Jewish Book Guide

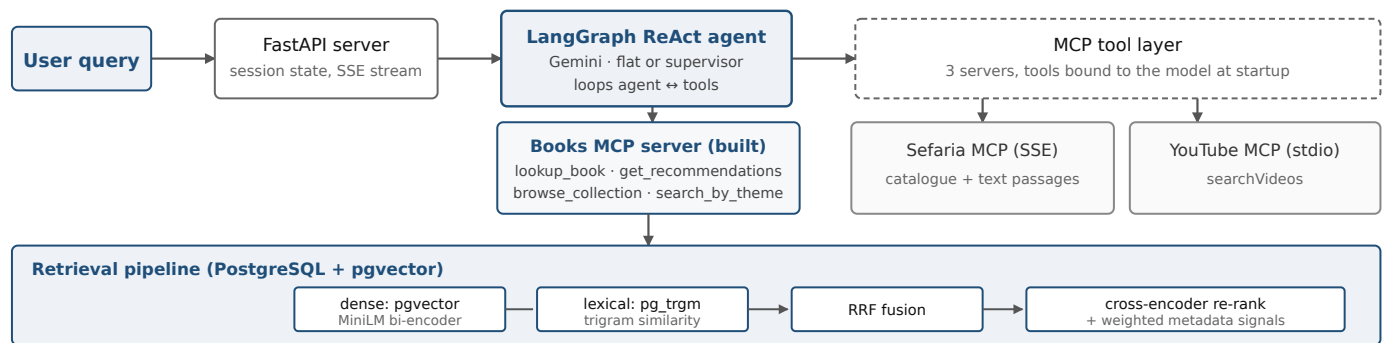
A conversational AI agent that recommends, looks up, and explains classical Jewish texts — built for readers who want to know what to study next, and as a working demonstration of agent orchestration, hybrid retrieval, and MCP server development.

Repo github.com/bennymestel/jewish-book-guide | **Live demo** jewish-book-guide.australia-southeast1.run.app

WHAT IT DOES

You chat with it. Say you enjoyed *Mesillat Yeshtarim* and it returns ranked recommendations from a curated 46-book collection of Chasidut, Musar, and Jewish Thought, each labelled with author and a 1–5 difficulty level. Ask about a theme like *teshuvah*, or for an intermediate Musar book, or to browse what's in the collection, and it picks the right retrieval tool for the question; results render as an interactive card grid rather than plain text. For anything outside the curated set it queries the Sefaria library directly, can pull the actual opening passage of a book, and can find introductory YouTube shiurim on a title or topic. Replies stream token-by-token, conversation context is kept per session, and the agent is constrained to its domain — off-topic and prompt-injection attempts are declined and covered by eval cases.

ARCHITECTURE



- **Two interchangeable agent graphs.** `?mode=simple` is a hand-built LangGraph StateGraph ReAct loop where one Gemini agent sees every tool from all three MCP servers. `?mode=multi` is a supervisor that sees only three `consult_*` tools, each wrapping a full specialist ReAct agent scoped to one server; the wrappers are async so independent consults run concurrently. Same state shape, so session handling is identical; a UI toggle switches between them. Measured on this eval set, the multi-agent graph did not improve quality and cost roughly 1.7× the latency — the honest result, kept in the repo rather than dropped.
- **MCP server, built not just consumed.** A standalone FastMCP server over streamable HTTP exposing four tools, two resources (`books://all`, plus a `ui://` MCP Apps card-grid view three of the tools declare) and two prompt templates. It talks to PostgreSQL directly and knows nothing about the agent layer.
- **Hybrid RAG, not just vector search.** Ingestion pulls book metadata from the Sefaria API, builds a composite profile per book (title, author, themes, description) and embeds it once with all-MiniLM-L6-v2 into a 384-dim pgvector column. `search_by_theme` runs dense and pg_trgm lexical arms, fuses them with Reciprocal Rank Fusion (k tuned to 10 for this corpus size, not the usual 60), then re-ranks with an mxbai cross-encoder. `get_recommendations` averages the seed books' embeddings for retrieval, widens the pool lexically when the user's own wording is available, then re-ranks on a weighted blend of cosine similarity, category/subcategory match, theme overlap, difficulty distance, and cross-encoder relevance.
- **Eval harness with real assertions.** 14 end-to-end cases (single- and multi-turn) run through the live graph and are graded on five axes: required tool called, tool arguments correct, every recommended title resolving to a real DB row, difficulty ceilings respected, and an LLM-as-judge on scope and faithfulness only — deliberately not on facts, which the deterministic checks own. For the multi-agent mode the harness reconstructs nested specialist tool calls from the event stream so trajectory checks still apply. Runs offline as a table or as a tracked LangSmith experiment, from one shared set of case definitions.

Stack — Python 3.11 · LangGraph + LangChain · Google Gemini · FastAPI (streaming SSE) · MCP (FastMCP server built; Sefaria over SSE and YouTube over stdio consumed) · PostgreSQL 17 + pgvector + pg_trgm · sentence-transformers (all-MiniLM-L6-v2 bi-encoder, mxbai-rerank-xsmall-v1 cross-encoder) · LangSmith tracing and experiments · Docker Compose (db → MCP → API, health-gated) · Cloud Run + Supabase, secrets in Secret Manager · pytest (39 unit tests) in GitHub Actions CI

Production concerns handled in code, not just described: bounded LLM/MCP timeouts, a daily request circuit breaker and quota-error messaging on the public demo, a /ready probe that touches both the DB and the tool count, model warm-up at MCP startup, and CORS locked shut by default.